

第25章

辨識手寫數字

25-1：認識KNN演算法

- KNN全名是K-Nearest Neighbor，可以翻譯為K最近鄰演算法。
- KNN (k-nearest neighbors)! 望文生義，就是物以類聚，如果你的鄰居10戶有8戶是有錢人，那你十之八九也是有錢人! KNN用途很廣，適用於離散型資料，也適用於連續型資料。! 那K是什麼呢? K代表著一個常數，也就是多少個鄰居算一類。
- KNN的運作模式
- 找到距離最近的K個鄰居→進行投票→決定類別
- 主要運作模式分為3步驟:
 - 步驟1. 計算每個點之間的距離 (歐幾里德距離)
 - 步驟2. 用K值決定鄰居數目，並進行投票 (在連續型資料中，則是計算平均數)
 - 步驟3. 以投票結果決定類別

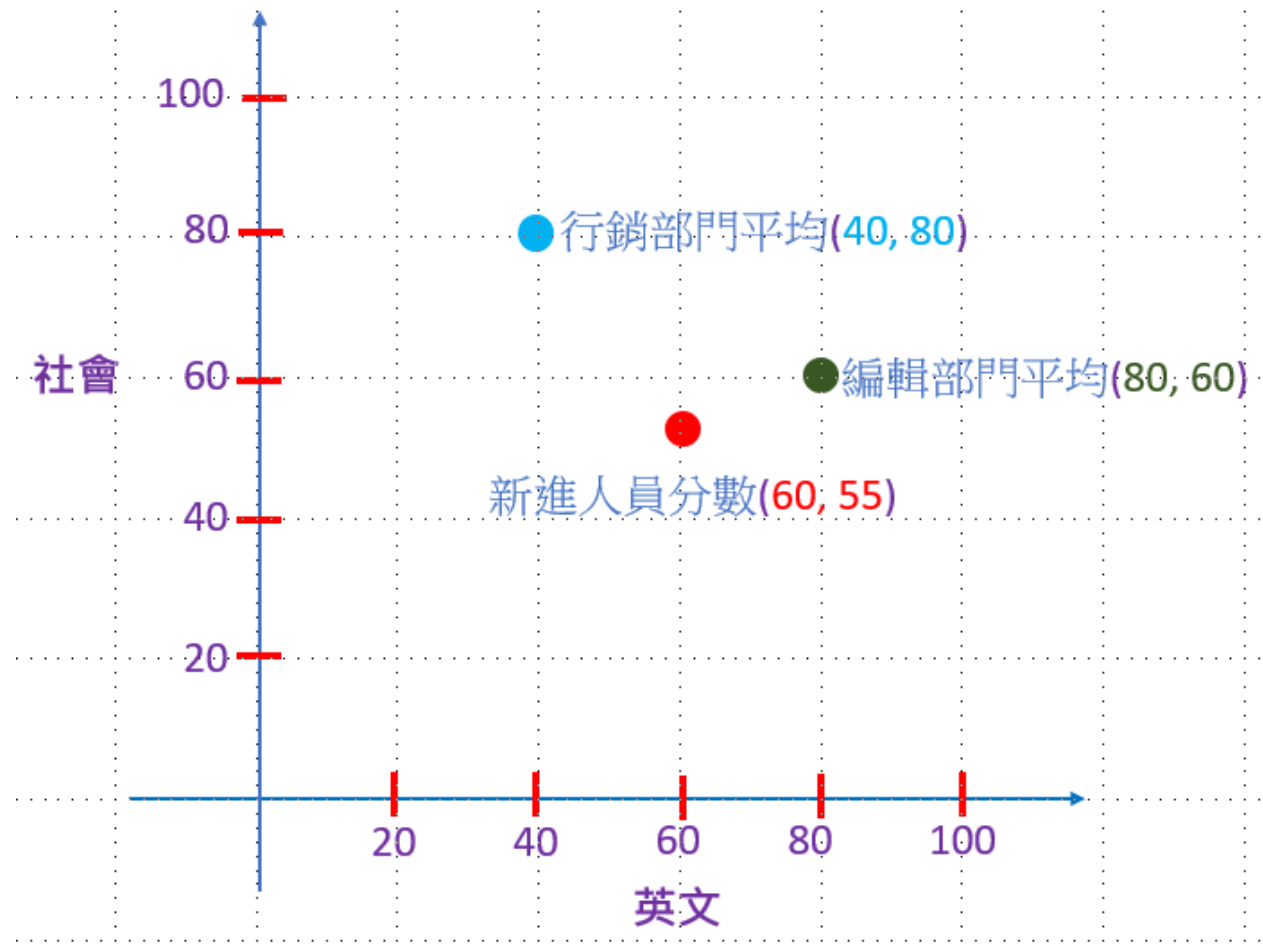


如果我們選擇 $K=3$ ，那我們就會以距離最接近的3個點當作鄰居，並看這3個鄰居當中有多少個鄰居是屬於哪一種類別，並以數量最多的類別取勝。以下圖來說，當 $K=3$ 時，因為藍色水滴有2個，黃色倒藍水滴只有1個，因此我們會判定星星為藍色水滴。

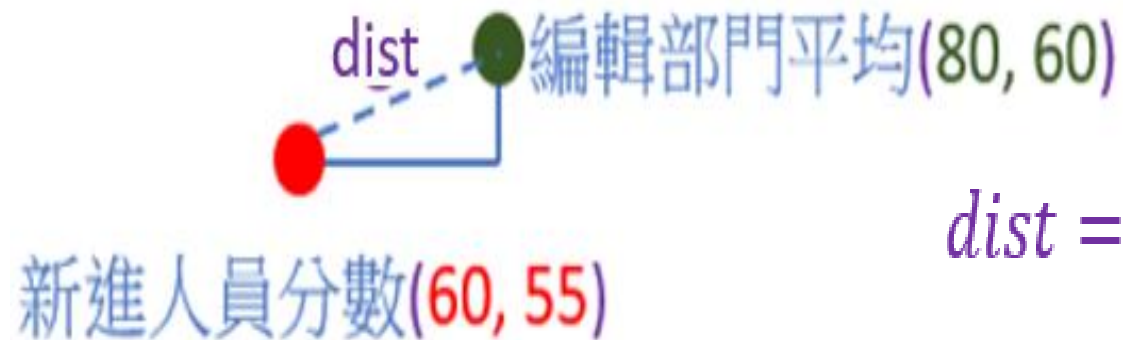


當 $K=5$ 時，我們則會判定星星為黃圈倒藍水滴，因為黃圈倒藍水滴有3個，大於藍水滴的2個。

25-1-1：數據分類的基礎觀念

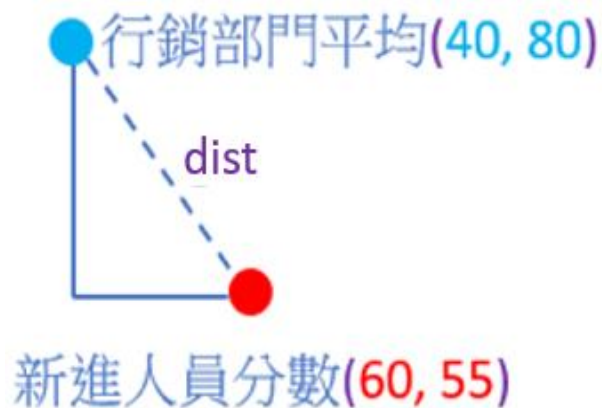


□ 計算新進人員分數和編輯部門平均分數的距離 使用畢氏定理



$$dist = \sqrt{(80 - 60)^2 + (60 - 55)^2} = \sqrt{425} = 20.6115$$

□ 計算新進人員分數和行銷部門平均分數的距離

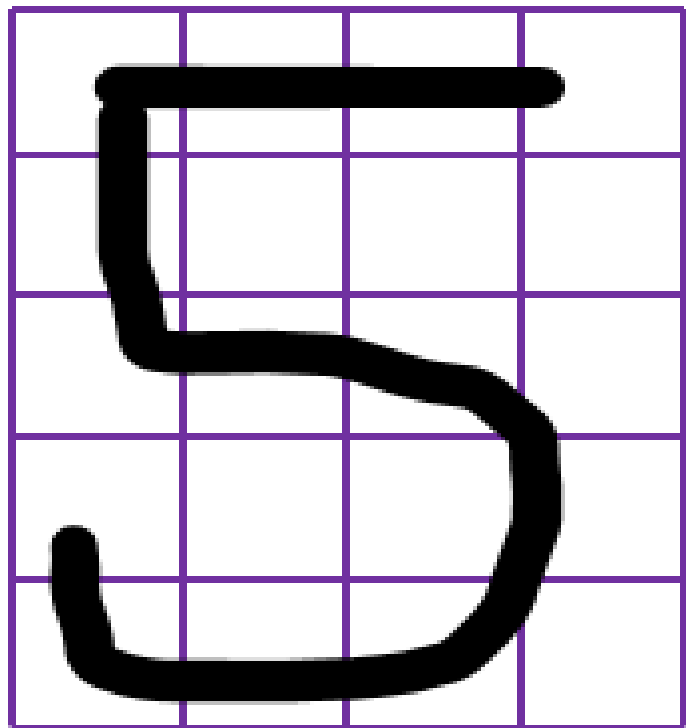


$$dist = \sqrt{(40 - 60)^2 + (80 - 55)^2} = \sqrt{1025} = 32.0156$$

□ 結論：

因新進人員的性向測驗分數較接近編輯部門，所以較適合進入編輯部門

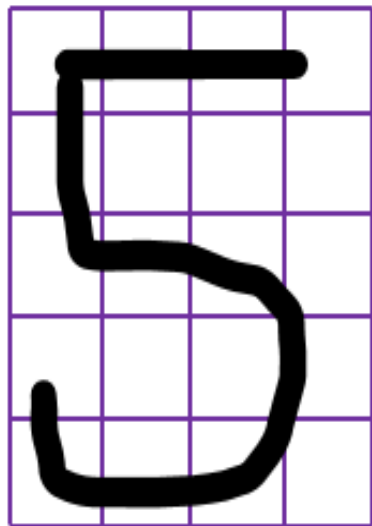
25-1-2：手寫數字的特徵



□左邊數字有20個方塊，如果將每個方塊拆成10X10個像素點，將手寫數字所佔據像素點數當作數字5的特徵。

- 第1個小方塊：18
- 第2個小方塊：30
- 第3個小方塊：30
- 第4個小方塊：6
- 第5個小方塊：30
- 第6個小方塊：0
- ...

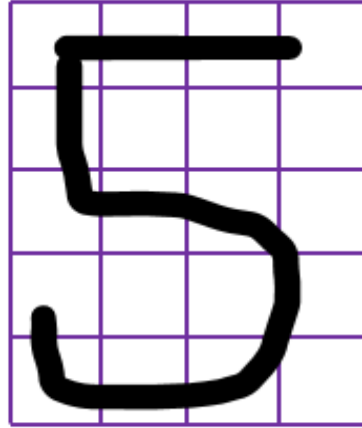
- 依據上述觀念我們可以得到上述數字5的相關資料如下：



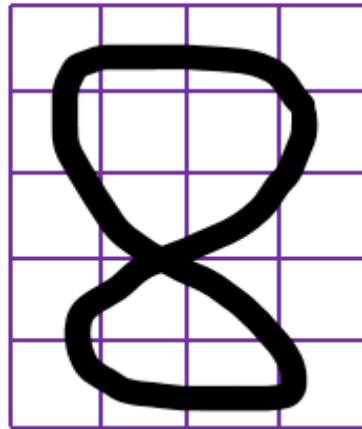
18	30	30	6
30	0	0	0
12	30	36	5
6	0	6	26
32	30	40	5

- 如果我們將上述數字轉成串列表示，可以得到下列結果。
- [18,30,30,6,30,0,0,0,12,30,36,5,6,0,6,26,32,30,40,5]
- 上述串列就可以視為是手寫數字5的特徵值。

25-1-3：不同數字特徵值的比較

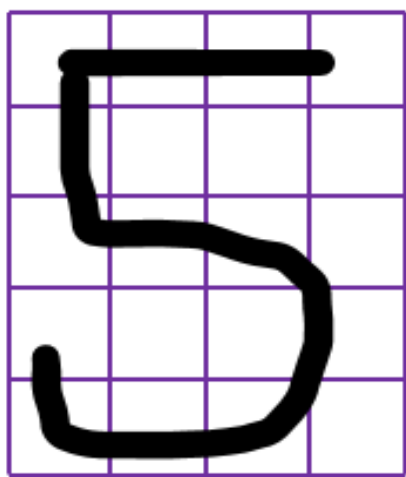


18	30	30	6
30	0	0	0
12	30	36	5
6	0	6	26
32	30	40	5

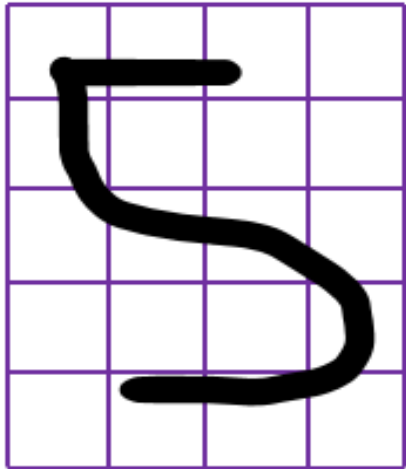


20	30	30	12
34	0	2	36
12	32	40	6
12	35	40	4
12	32	46	20

25-1-4：手寫數字分類原理



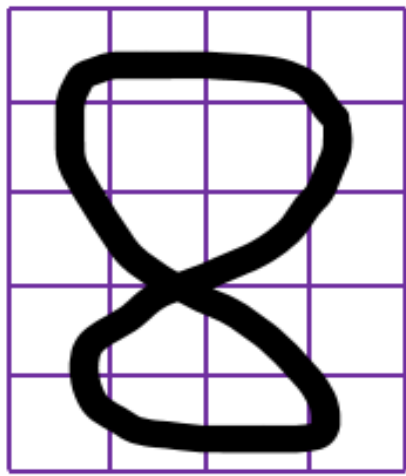
18	30	30	6
30	0	0	0
12	30	36	5
6	0	6	26
32	30	40	5



18	30	6	0
30	0	0	0
10	30	36	10
0	0	0	42
0	26	32	8

需要辨識的手寫數字

數字特徵



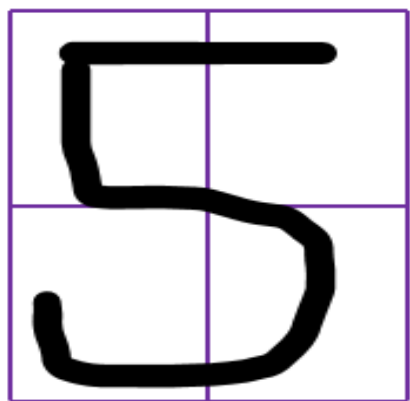
20	30	30	12
34	0	2	36
12	32	40	6
12	35	40	4
12	32	46	20

$$dist = \sqrt{(18 - 18)^2 + (30 - 30)^2 + \dots (5 - 8)^2}$$

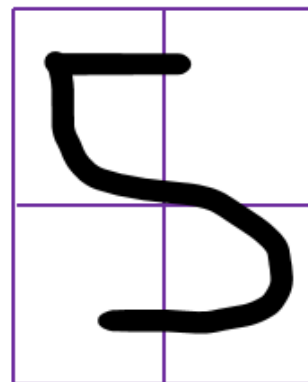
$$dist = \sqrt{(20 - 18)^2 + (30 - 30)^2 + \dots (20 - 8)^2}$$

經過計算距離比較小判斷待辨識數字屬於5或8

25-1-5：簡化特徵比較



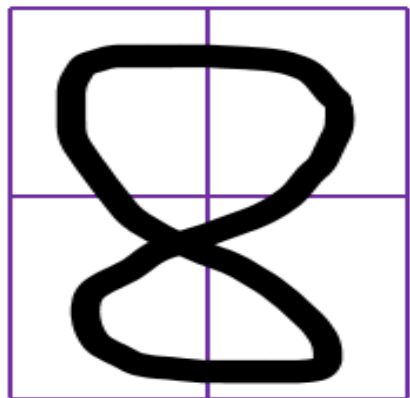
30	16
22	28



30	12
8	32

需要辨識的手寫數字

數字特徵



34	34
32	32

$$dist = \sqrt{(30 - 30)^2 + (16 - 12)^2 + (22 - 8)^2 + (28 - 32)^2} \approx 15.1$$

$$dist = \sqrt{(34 - 30)^2 + (34 - 12)^2 + (32 - 8)^2 + (32 - 32)^2} \approx 32.8$$

經過計算右邊數字距離與左邊5的數字距離比較近，
所以將需要辨識的手寫數字分類為5

25-2：認識Numpy與KNN演算法相關的知識

- 25-2-1：Numpy的seed()函數
- Randint()函數:當建立指定維度的陣列，每次執行皆會產生不同元素的陣列。
- 程式實例1.py：建立含5個元素的一維陣列(陣列元素是整數)或稱1 x 5陣列，和建立5 x 1的矩陣(或是稱二維陣列)。

```
import numpy as np

data1 = np.random.randint(0, 10, size = 5)
print(f"陣列外形 = {data1.shape}")
print(f"輸出陣列 = {data1}")
print(f"data1[0] = {data1[0]}")
data2 = np.random.randint(0, 10, size = (5, 1))
print(f"矩陣外形 = {data2.shape}")
print(f"輸出矩陣 = \n{data2}")
print(f"data2[0] = {data2[0]}")
print(f"data2[0,0] = {data2[0,0]}")
```

```
===== RESTART: ===== RESTART
陣列外形 = (5,) 陣列外形 = (5,)
輸出陣列 = [1 8 9 6 3] 輸出陣列 = [8 2 7 1 4]
data1[0] = 1 data1[0] = 8
矩陣外形 = (5, 1) 矩陣外形 = (5, 1)
輸出矩陣 = 輸出矩陣 =
[[8] [[6]
 [4] [1]
 [3] [9]
 [3] [3]
 [1]] [0]]
data2[0] = [8] data2[0] = [6]
data2[0,0] = 8 data2[0,0] = 6
>>>
```

- Randint()每次執行都產生不同的數字
- Random.Seed(**n**), **n** 是種子值(可以視為32位元無號整數),之後產生固定的隨機數
- 程式實例2.py：使用np.random.seed(5)，未來可以產生固定的隨機數。

```
# ch25_2.py
import numpy as np
np.random.seed(5)
data1 = np.random.randint(0, 10, size = 5)
print(f"陣列外形 = {data1.shape}")
print(f"輸出陣列 = {data1}")
print(f"data1[0] = {data1[0]}")
data2 = np.random.randint(0, 10, size = (5, 1))
print(f"矩陣外形 = {data2.shape}")
print(f"輸出矩陣 = \n{data2}")
print(f"data2[0] = {data2[0]}")
print(f"data2[0,0] = {data2[0,0]}")
```

```
===== RESTART: D:/Of
陣列外形 = (5,)
輸出陣列 = [3 6 6 0 9]
data1[0] = 3
矩陣外形 = (5, 1)
輸出矩陣 =
[[8]
 [4]
 [7]
 [0]
 [0]]
data2[0] = [8]
data2[0,0] = 8
>>>
===== RESTART: D:/Of
陣列外形 = (5,)
輸出陣列 = [3 6 6 0 9]
data1[0] = 3
矩陣外形 = (5, 1)
輸出矩陣 =
[[8]
 [4]
 [7]
 [0]
 [0]]
data2[0] = [8]
data2[0,0] = 8
```

25-2-2 : Numpy的ravel()函數

- `ravel()` 把多維陣列轉為一維陣列
- 程式實例3.py：將二維陣列轉為一維陣列。

```
import numpy as np

data = np.random.randint(0, 10, size = (5, 1))
print(f"輸出二維陣列 = \n{data}")
print(f"轉成一維陣列 = \n{data.ravel()}")
```

```
===== RESTART: D:/OpenCV_Python/ch25/ch25_3.py =====
輸出二維陣列 =
[[9]
 [5]
 [2]
 [5]
 [6]]
轉成一維陣列 =
[9 5 2 5 6]
```

25-2-3：數據分類

- 程式實例：建立一個5 x 2的二維陣列，然後建立一個1 x 5的分類陣列(也可以想成是分類器)索引，分類器的值是0或1，如果值是0則將相對應的5 x 2陣列索引內容歸到紅色(red)類，如果值是1則將相對應的5 x 2陣列索引內容歸到藍色(blue)類。

```
import numpy as np

np.random.seed(1)
trains = np.random.randint(0, 10, size = (5, 2))
print(f"列出二維陣列 \n{trains}")

np.random.seed(5)
# 建立分類，未來 0 代表 red, 1 代表 blue
labels = np.random.randint(0, 2, (5, 1))
print(f"列出顏色分類陣列 \n{labels}")

# 列出 0 代表的紅色
red = trains[labels.ravel() == 0]
print(f"輸出紅色的二維陣列 \n{red}")
print(f"配對取出 \n{red[:, 0], red[:, 1]}")

# 列出 1 代表的藍色
blue = trains[labels.ravel() == 1]
print(f"輸出藍色的二維陣列 \n{blue}")
print(f"配對取出 \n{blue[:, 0], blue[:, 1]}")
```

```
===== RESTART: D:/OpenCV_
列出二維陣列
[[5 8]
 [9 5]
 [0 0]
 [1 7]
 [6 9]]
列出顏色分類陣列
[[1]
 [0]
 [1]
 [1]
 [0]]
輸出紅色的二維陣列
[[9 5]
 [6 9]]
配對取出
(array([9, 6]), array([5, 9]))
輸出藍色色的二維陣列
[[5 8]
 [0 0]
 [1 7]]
配對取出
(array([5, 0, 1]), array([8, 0, 7]))
```

25-2-4：建立與分類30筆訓練數據

- 程式實例：假設(x, y)代表數據的特徵，這個程式會繪製30筆訓練數據，然後也會用隨機數產生0或1的值，0或1分別標記為紅色方塊或是藍色三角形，最後繪製此圖。

Generate a 2 x 4 array of ints between

```
import numpy as np
import matplotlib.pyplot as plt

num = 30                                # 數據數量
np.random.seed(5)
trains = np.random.randint(0, 100, size = (num, 2))  # 產生介於0~100的30X2陣列
np.random.seed(1)
# 建立分類，未來 0 代表 red, 1 代表 blue
labels = np.random.randint(0,2,(num,1))
# 列出紅色方塊訓練數據
red = trains[labels.ravel() == 0]
plt.scatter(red[:, 0], red[:, 1], s=50, c='r', marker='s')    # c='r',red;marker='s' ,square，方形
# 列出藍色三角形訓練數據
blue = trains[labels.ravel() == 1]
plt.scatter(blue[:, 0], blue[:, 1], s=50, c='b', marker='^')  # S點的大小,50是繪圖點大小;marker='^',triangle，三角形

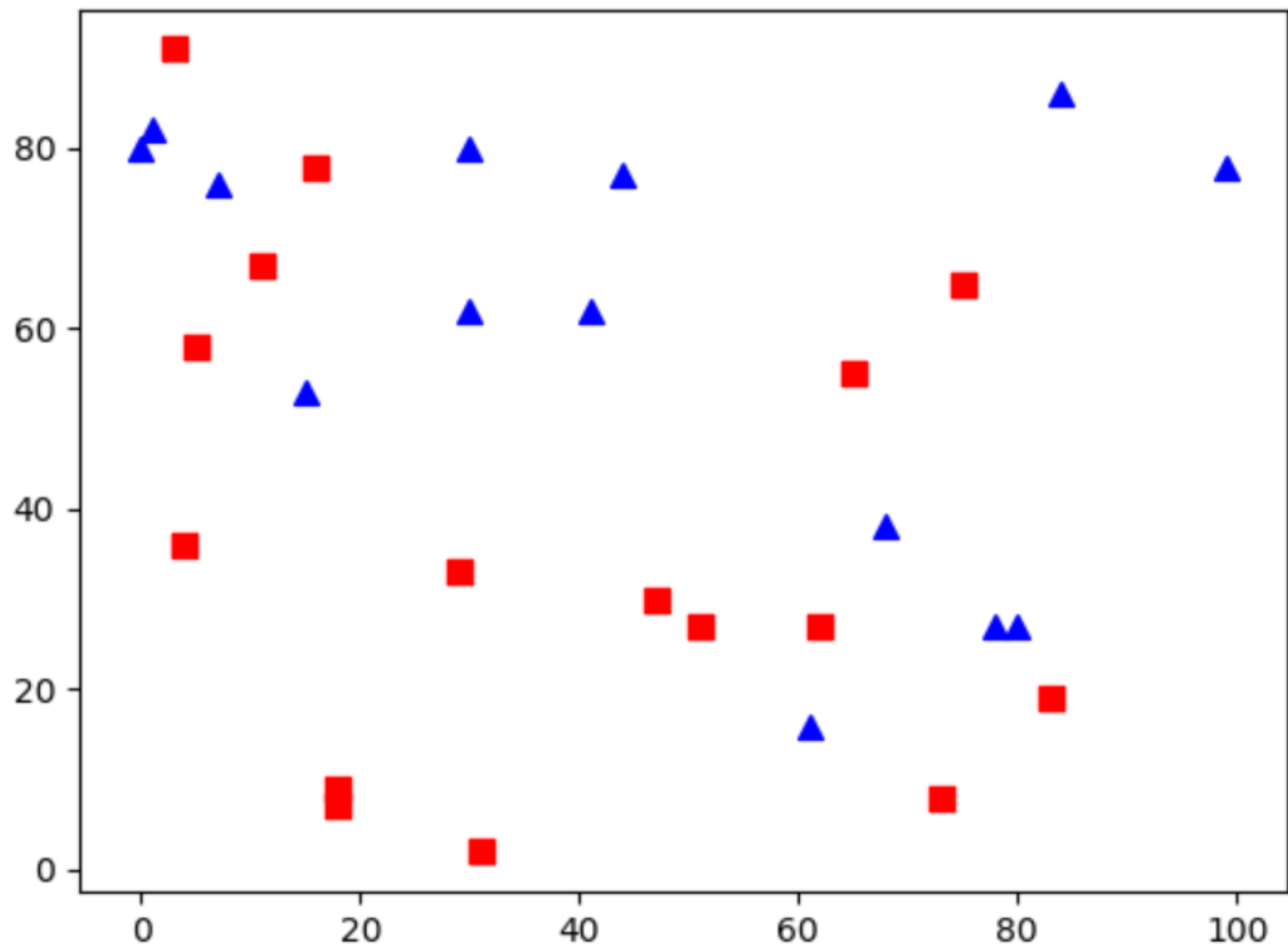
plt.show()
```

```
.randint(5, size=(
2, 1]), # random
2, 0]])
```

- `plt.scatter(x, y, size, color, marker)`
- `x, y`：分別是 `x` 軸與 `y` 軸的座標。
- `size`：點的大小（例如 `50` 表示點的面積）
- `color`：顏色，例如 `'r'` 是 `red`，`'b'` 是 `blue`
- `marker`：點的形狀。

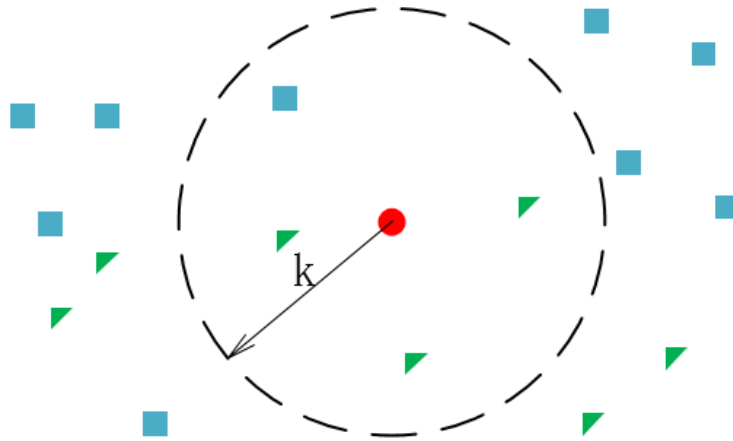
Marker 參數設定:

參數	圖形說明
's'	square (方形)
'^'	triangle_up (上三角形)
'o'	circle (圓形)
'v'	triangle_down (下三角形)
'x'	X 符號
'+'	加號
'd'	diamond (菱形)
'p'	五邊形
'*'	星號



25-3 : OpenCV的KNN演算法函數

- 所謂最鄰近演算法，即給定一個已訓練資料集，對新的輸入實例，在訓練資料集中找到與該實例最鄰近（注：衡量鄰近的標準以具體選取的某個特徵而言，例如下面示意圖中使用的特徵為歐式距離）的K個實例，這K個實例的多數屬於某個類，則判定該輸入實例同屬此類。
- 如下圖所示：訓練者取k值，計算以歐氏距離k為半徑的圓內其他類別的個數，圖中中心小紅點以k為半徑的圓內三角形個數最多，則判定中心小紅點為三角形。



25-3 : OpenCV的KNN演算法函數

你是誰，要看你旁邊最近的人大多是誰

- 使用KNN需先使用`cv2.ml.KNearest_create()`建立knn物件

建立一個 KNN 分類器

先準備一個「會找鄰居來判斷類別」的模型。

- `knn = cv2.ml.KNearest_create()`

- 使用**TRAIN()**訓練數據: `knn.train(train, cv2.ml.ROW_SAMPLE, labels)` 把訓練資料交給 KNN。

- `train` 訓練數據 訓練資料，例如很多筆點座標、影像特徵、顏色特徵等。
- `cv2.ml.ROW_SAMPLE` 將陣列的長度視為1列 代表「每一列是一筆資料」
- `labels` 分類結果 每筆訓練資料的答案，也就是分類結果

- `knn.train()`函數執行成功將回傳True

```
labels = [  
    0,  
    0,  
    1  
]
```

```
train = [  
    [1, 2],  
    [2, 3],  
    [8, 9]  
]
```

有 3 筆資料，每筆資料有 2 個特徵。

請 KNN 判斷 test 測試資料屬於哪一類

- 測試數據分類: `ret, results, neighbours, dist = knn.findNearest(test, k=n)`

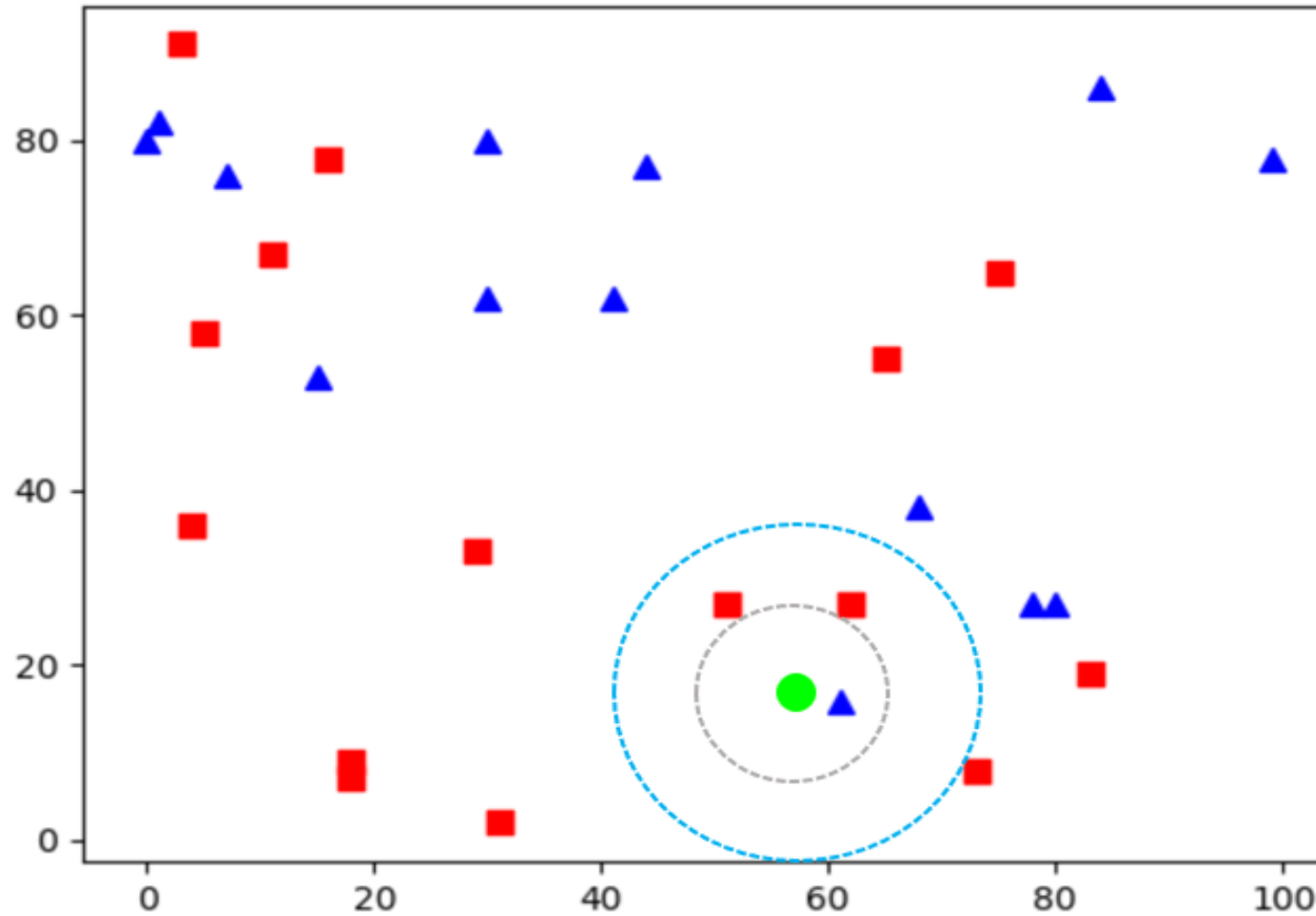
- `results` 為數據分類的結果; 代表分類結果的回傳值，通常和 `results` 內容相同或接近

- `Neighbours` : 找到的最近鄰居類別; `neighbours = [[0, 0, 1]]` → 代表最近的 3 個鄰居類別分別是 0、0、1。

- `dist` 目前相鄰數據的距離; `dist = [[1.2, 2.5, 4.0]]`; 代表距離越小，資料越接近。

- `findNearest()` 函數所傳遞的參數 `test` 是測試數據，`k` 是設定依據多少組數據作判斷，如果 `k = 1` 代表是 1-KNN 演算法，如果 `k = 3` 代表是 3-KNN 演算法，其他依此類推。

- KNN演算法分類器的初始化，傳入一個訓練資料集，以及對應的label。給它一個測試資料，讓它來進行分類。OpenCV中使用knn.findNearest()函數
 - 參數1：測試資料
 - 參數2：k的值
- 返回值1：由kNN演算法計算得到的測試資料的類別標誌（0 或 1）。如果你想使用最近鄰演算法，只需要將k設置1，k就是最近鄰的數目
- 返回值2：k的最近鄰居的類別標誌
- 返回值3：每個最近鄰居到測試資料的距離



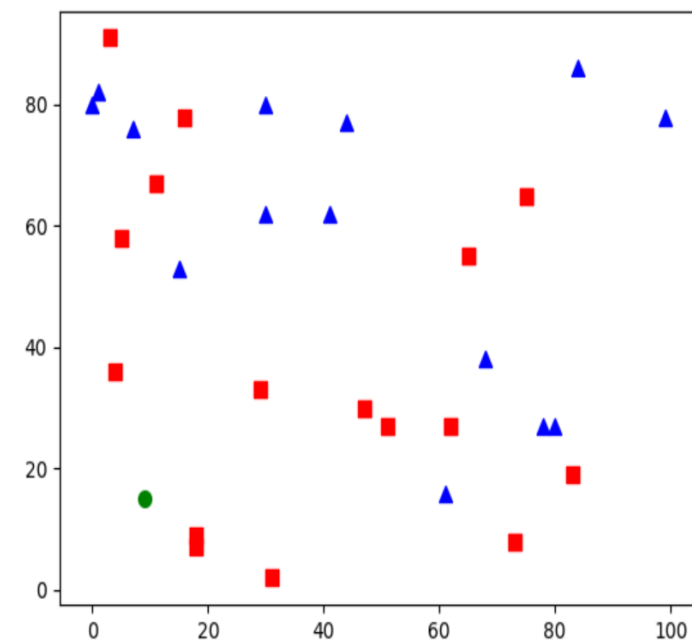
- 在上述情況如果 $k = 1$ ，則測試數據將分類藍色三角形類別，但是如果 $k = 3$ 則測試數據將分類紅色方塊類別。

- 程式實例：設定 $k = 3$ ，執行 KNN 演算法的測試數據分類，將測試數據用綠色圓表示。

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

num = 30 # 數據數量
np.random.seed(5)
# 建立訓練數據 train, 需轉為 32位元浮點數
trains = np.random.randint(0, 100, size = (num, 2)).astype(np.float32)
np.random.seed(1)
# 建立分類, 未來 0 代表 red, 1 代表 blue
labels = np.random.randint(0,2,(num,1)).astype(np.float32)
# 列出紅色方塊訓練數據
red = trains[labels.ravel() == 0]
plt.scatter(red[:,0],red[:,1],50,'r','s') # 50是繪圖點大小
# 列出藍色三角形訓練數據
blue = trains[labels.ravel() == 1]
plt.scatter(blue[:,0],blue[:,1],50,'b','^') # 50是繪圖點大小
# test 為測試數據, 需轉為 32位元浮點數
np.random.seed(10)
test = np.random.randint(0, 100, (1, 2)).astype(np.float32)
plt.scatter(test[:,0],test[:,1],50,'g','o') # 50大小的綠色圓
# 建立 KNN 物件
knn = cv2.ml.KNearest_create()
knn.train(trains, cv2.ml.ROW_SAMPLE, labels) # 訓練數據
# 執行 KNN 分類
ret, results, neighbours, dist = knn.findNearest(test, k=3)
print(f"最後分類 result = {results}")
print(f"最近鄰3個點的分類 neighbours = {neighbours}")
print(f"與最近鄰的距離 distance = {dist}")

plt.show()
```



```
最後分類 result = [[0.]]
最近鄰3個點的分類 neighbours = [[0. 0. 0.]]
與最近鄰的距離 distance = [[117. 145. 466.]]
```

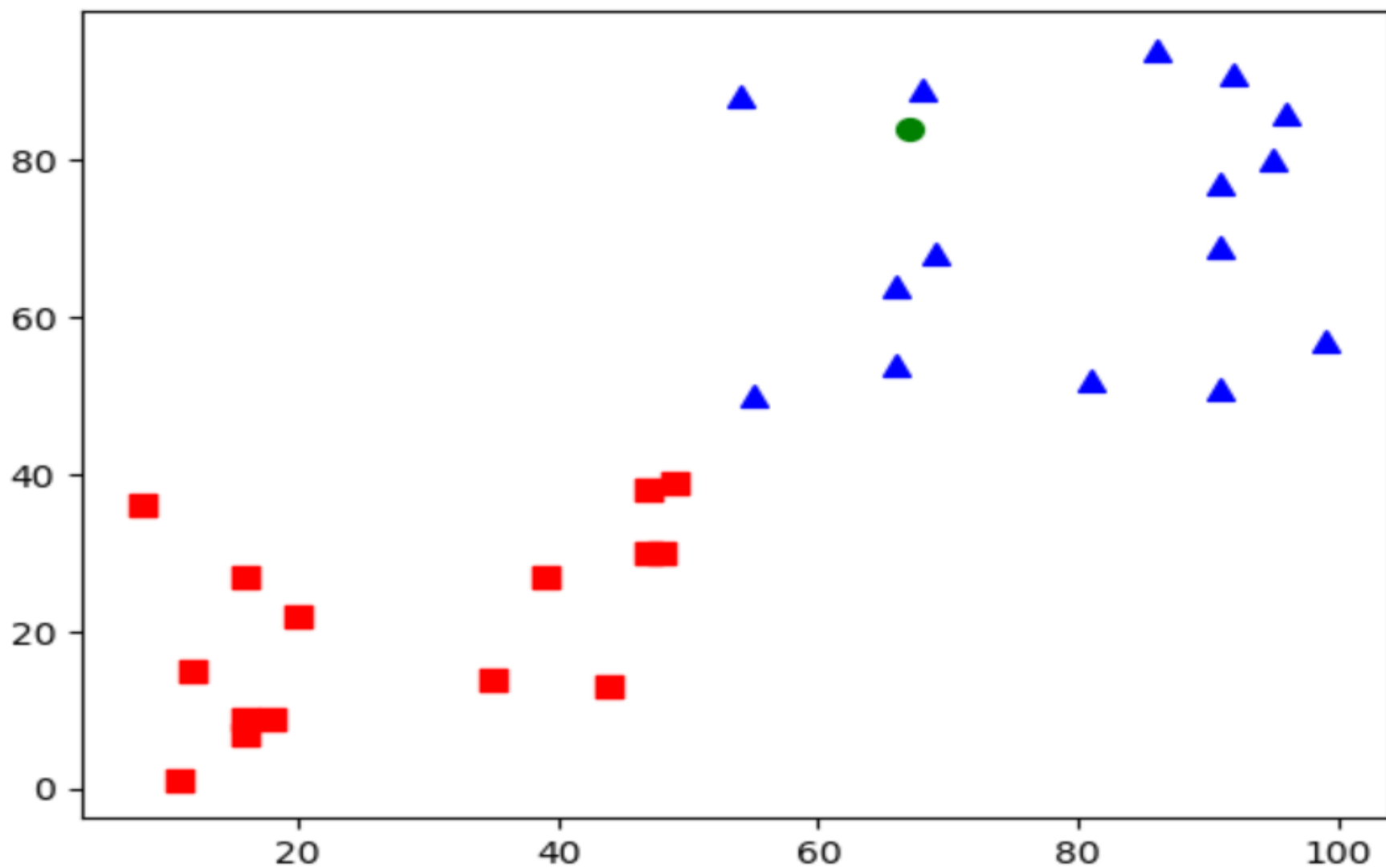
25-3-2：更常見的分類

- 程式實例：使用0-50 與, 51-100 之間隨機數歸一類, 建立兩個群聚的類別，然後使用`np.vstack( )`將兩個群聚的類別合併，最後再測試數據，種子值設定為`np.random.seed(8)`，所以可以得到不同的測試數據位置。

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

num = 30 # 數據數量
np.random.seed(5)
# 建立 0 - 50 間的訓練數據 train0, 需轉為 32位元浮點數
train0 = np.random.randint(0, 50, (num // 2, 2)).astype(np.float32)
# 建立 50 - 100 間的訓練數據 train1, 需轉為 32位元浮點數
train1 = np.random.randint(50, 100, (num // 2, 2)).astype(np.float32)
trains = np.vstack((train0, train1)) # 合併訓練數據
# 建立分類, 未來 0 代表 red, 1 代表 blue
label0 = np.zeros((num // 2, 1)).astype(np.float32)
label1 = np.ones((num // 2, 1)).astype(np.float32)
labels = np.vstack((label0, label1))
# 列出紅色方塊訓練數據
red = trains[labels.ravel() == 0]
plt.scatter(red[:, 0], red[:, 1], 50, 'r', 's') # 50是繪圖點大小
# 列出藍色三角形訓練數據
blue = trains[labels.ravel() == 1]
plt.scatter(blue[:, 0], blue[:, 1], 50, 'b', '^') # 50是繪圖點大小
# test 為測試數據, 需轉為 32位元浮點數
np.random.seed(8)
test = np.random.randint(0, 100, (1, 2)).astype(np.float32)
plt.scatter(test[:, 0], test[:, 1], 50, 'g', 'o') # 50大小的綠色圓
# 建立 KNN 物件
knn = cv2.ml.KNearest_create()
knn.train(trains, cv2.ml.ROW_SAMPLE, labels) # 訓練數據
# 執行 KNN 分類
ret, results, neighbours, dist = knn.findNearest(test, k=3)
print(f"最後分類 result = {results}")
print(f"最近鄰3個點的分類 neighbours = {neighbours}")
print(f"與最近鄰的距離 distance = {dist}")

plt.show()
```



```
最後分類          result = [[1.]]  
最近鄰3個點的分類 neighbours = [[1. 1. 1.]]  
與最近鄰的距離    distance = [[ 26. 185. 260.]]
```

25-4：有關手寫數字識別的Numpy基礎知識

- 25-4-1：vsplit()垂直方向分割數據
- np.vsplit(ary, indices_or_section)
- 程式實例：使用vsplit()函數，可以ROW分割數據。

```
import numpy as np

data = np.arange(16).reshape(4,4)
print(f"data = \n {data}")
print(f"split = \n{np.vsplit(data,2)}")
```

```
data =
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
split =
[array([[0, 1, 2, 3],
       [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
       [12, 13, 14, 15]])]
```

- 程式實例：使用`vsplit( )`函數，將三維度陣列做分割。

```
===== RESTART: D:\OpenCV_Python\ch25\ch25_9.py =====  
data =  
  [[[0 1]  
    [2 3]]  
  
  [[4 5]  
   [6 7]]]  
split =  
[array([[0, 1],  
        [2, 3]]), array([[4, 5],  
        [6, 7]])]
```

25-4-2 : hsplit()水平方向分割數據

- `np.hsplit(ary, sections)`
- 程式實例：使用`hsplit( )`函數，將資料依COLUMN方向分割。

```
import numpy as np

data = np.arange(16).reshape(4,4)
print(f"data = \n {data}")
print(f"split = \n{np.hsplit(data,2)}")
```

```
data =
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
split =
[array([[ 0,  1],
        [ 4,  5],
        [ 8,  9],
        [12, 13]]), array([[ 2,  3],
        [ 6,  7],
        [10, 11],
        [14, 15]])]
```

25-4-3：元素重複repeat()

- `np.repeat(a, repeat, axis)` 可以覆製陣列的元素
- A是陣列;
 - Repeat:是整數或整數陣列代表重覆次數
 - Axis:代表軸預設是0
- `repeat( )`函數的基礎應用。

```
import numpy as np

data = np.arange(3)
print(f"data = \n {data}")
x = np.repeat(data, 3)
print(f"After repeat = \n{x}")
```

```
data =
[0 1 2]
After repeat =
[0 0 0 1 1 1 2 2 2]
```

- 程式實例：repeat()函數進一步的應用。

axis=0 # 直向，沿著列 row 方向； **axis=1** # 橫向，沿著欄 column 方向

```
import numpy as np

data = np.array([[1,2],[3,4]])
print(f"data = \n {data}")
x1 = np.repeat(data, 3, axis=1)
print(f"After axis=1 repeat = \n{x1}")
x2 = np.repeat(data, 3, axis=0)
print(f"After axis=0 repeat = \n{x2}")
```

```
data =
[[1 2]
 [3 4]]
After axis=1 repeat =
[[1 1 1 2 2 2]
 [3 3 3 4 4 4]]
After axis=0 repeat =
[[1 2]
 [1 2]
 [1 2]
 [3 4]
 [3 4]
 [3 4]]
```

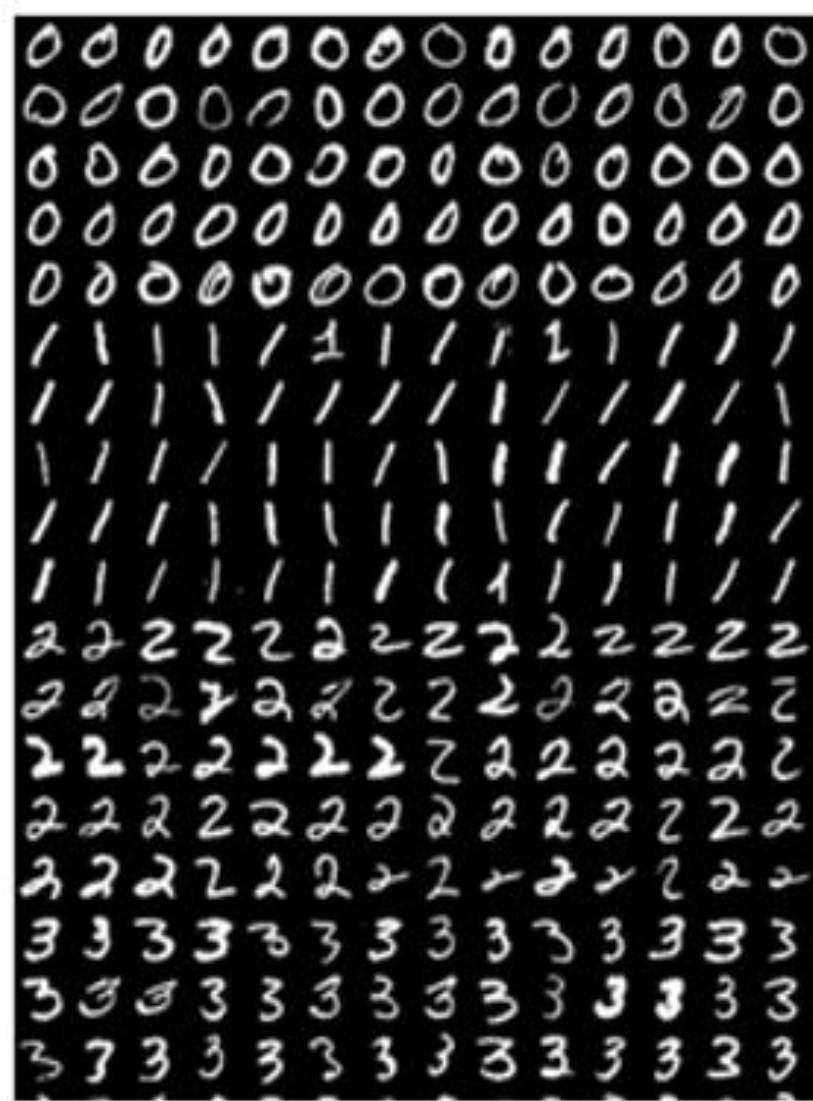
25-5：識別手寫數字

- 25-5-1：實際設計識別手寫數字
 - 檔案:digits.png 內含5000個手寫數字0~9數字, 每個數字重覆了500次
 - 影像訓練與測試數據, 可以影像拆解成將5000個數字影像
-
- digits.png 是一張包含 5000 張數字圖 (50 行 × 100 列)
 - 每個數字大小為 **20x20** 像素
 - 總共： $50 \times 100 = 5000$ 張手寫數字圖

Train Datasets

[illegible]

Test Datasets



- **程式實例**：使用 $k = 5$ 的KNN演算法，將digits.png數字影像檔案的前2500個當作樣本訓練數據和使用後2500個手寫數字當作測試數據，最後列出後2500個手寫數字辨識成功率。

```
import cv2
import numpy as np

img = cv2.imread('digits.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 將digits拆成 5000 張, 20 x 20 的數字影像
cells = [np.hsplit(row, 100) for row in np.vsplit(gray, 50)]
# 將 cells 轉成 50 x 100 x 20 x 20 的陣列
x = np.array(cells)
# 將數據轉為訓練數據 size=(2500,400)和測試數據 size=(2500,400)
train = x[:, :50].reshape(-1, 400).astype(np.float32) 把圖片變成20X20向量
test = x[:, 50:100].reshape(-1, 400).astype(np.float32)
# 建立訓練數據和測試數據的分類 labels -1 表示NumPy 會自動算2500 x 20 x 20= 2500 x 400
k = np.arange(10)
train_labels = np.repeat(k, 250)[:, np.newaxis]
test_labels = train_labels.copy()
# 最初始KNN或稱建立KNN物件，訓練數據、使用 k=5 測試KNN演算法
knn = cv2.ml.KNearest_create()
knn.train(train, cv2.ml.ROW_SAMPLE, train_labels)
ret, result, neighbours, dist = knn.findNearest(test, k=5)
# 統計辨識結果
matches = result==test_labels # 執行匹配
correct = np.count_nonzero(matches) # 正確次數
accuracy = correct * 100.0 / result.size # 精確度
print(f"測試數據辨識成功率 = {accuracy}")
```

25-5-2：儲存訓練和分類數據

- 儲存所得之數據
- `np.savez('name.npz', train=train, train_labels=train_labels)`
 - `name.npz`:要儲存的檔案名稱,可自訂
 - `Train`:訓練數據
 - `Train_labels`:分類數據
- 程式實例：儲存訓練數據到檔案`knn_digit.npz`。
 - `np.savez('knn_digit.npz', train=train, train_labels=train_labels)`
- 執行結果：這個程式的執行結果與`ch25_12.py`相同，不過在`ch25`資料夾可以看到`knn_digit.npz`檔案。

```
import cv2
import numpy as np

img = cv2.imread('digits.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 將digits拆成 5000 張, 20 x 20 的數字影像
cells = [np.hsplit(row, 100) for row in np.vsplit(gray, 50)]
# 將 cells 轉成 50 x 100 x 20 x 20 的陣列
x = np.array(cells)
# 將數據轉為訓練數據 size=(2500,400)和測試數據 size=(2500,400)
train = x[:, :50].reshape(-1, 400).astype(np.float32)
test = x[:, 50:100].reshape(-1, 400).astype(np.float32)
# 建立訓練數據和測試數據的分類 labels
k = np.arange(10)
train_labels = np.repeat(k, 250)[:, np.newaxis]
test_labels = train_labels.copy()
# 最初始KNN或稱建立KNN物件，訓練數據、使用 k=5 測試KNN演算法
knn = cv2.ml.KNearest_create()
knn.train(train, cv2.ml.ROW_SAMPLE, train_labels)
ret, result, neighbours, dist = knn.findNearest(test, k=5)
# 統計辨識結果
matches = result==test_labels # 執行匹配
correct = np.count_nonzero(matches) # 正確次數
accuracy = correct * 100.0 / result.size # 精確度
print(f"測試數據辨識成功率 = {accuracy}")
np.savez('knn_digit.npz', train=train, train_labels=train_labels)
```

25-5-3：下載訓練和分類數據

- with `np.load('knn.digit.npz')` as data:
- `train = data['train']`
- `train_labels = data['train_labels']`
-
- 程式實例：執行8.png影像測試，同時回應執行結果。

數字辨識演算法

1. 從 knn_digit.npz 檔案中載入訓練資料：
 - train: 所有手寫數字的訓練影像特徵
 - train_labels: 對應的數字標籤 (0~9)
2. 讀取欲辨識的圖像檔 (例如 '8.png')
 - 將影像轉為灰階模式
3. 顯示這張要辨識的影像 (可略過顯示步驟)
4. 將影像縮放為 20x20 大小 (KNN 訓練用的標準尺寸)
 - 並攤平成 1x400 的向量 (20x20 = 400 像素點)
5. 將圖像資料轉為 float32 型態, 以便 OpenCV 處理
6. 建立 KNN 分類器 (使用 OpenCV 提供的 KNearest 演算法)
7. 將訓練資料與標籤餵進 KNN 模型進行訓練
8. 用 KNN 對測試圖像做預測 (k = 5) :
 - 找出與此圖像最相近的 5 筆訓練資料
 - 由這 5 筆的「多數分類結果」決定最終分類
9. 印出辨識結果 (例如: 這張圖被判斷為「8」)
10. 等待使用者按鍵後關閉視窗

```
import cv2
import numpy as np

# 下載數據
with np.load('knn_digit.npz') as data:
    train = data['train']
    train_labels = data['train_labels']
# 讀取數字影像
test_img = cv2.imread('8.png', cv2.IMREAD_GRAYSCALE)
cv2.imshow('img', test_img)
img = cv2.resize(test_img, (20, 20)).reshape((1, 400))
test_data = img.astype(np.float32) # 將資料轉成float32
# 最初化KNN或稱建立KNN物件，訓練數據、使用 k=5 測試KNN演算法
knn = cv2.ml.KNearest_create()
knn.train(train, cv2.ml.ROW_SAMPLE, train_labels)
ret, result, neighbours, dist = knn.findNearest(test_data, k=5)
print(f"識別的數字是 = {int(result[0,0])}")

cv2.waitKey(0)
cv2.destroyAllWindows()
```

識別的數字是 = 8



題目練習

- 讀取這張影像, 並顯示識別的數字為多少?